

# 汽车微处理器原理与应用

## 实 验 报 告

### 基本信息

|      |             |      |            |
|------|-------------|------|------------|
| 实验名称 | 汽车档位模拟器设计   | 学生姓名 | 刘潇霞        |
| 学 号  | 23071140208 | 班 级  | 汽服 2302B 班 |
| 组 号  | 1           | 指导教师 | 刘兆骐        |
| 实验日期 | 2026. 6. 4  | 提交日期 | 2026. 6. 6 |

### 一、团队信息与分工（5 分）

#### 1.1 团队成员基本信息

| 组号 | 角色 | 姓名  | 学号          |
|----|----|-----|-------------|
| 1  | 组长 | 刘潇霞 | 23071140208 |
| 1  | 组员 | 王舒乐 | 23071140204 |

#### 1.2 个人分工说明

我在本次汽车档位模拟器实验中承担实验主体工作。实验实操阶段，使用 STM32CubeMX 完成芯片引脚与外设初始化配置，负责外部中断驱动、DWT 精准消抖、PRND 档位状态机程序编写，在 MDK-ARM 中完成工程编译、排错调试，熟练使用 ATK-XISP 工具通过 ISP 模式完成固件烧录，排查解决程序档位错乱、烧录连接失败等关键问题，率先调试出档位切换的完整实验现象。实

验收尾时，和组员一同商讨实验报告的大纲、各部分写作要点，梳理实验原理、故障分析等核心内容。

### 1.3 团队协作过程记录

本次实验小组共两人，围绕汽车档位模拟器设计开展分工协作。实验前期，组长统筹整体实验方案，承担实验主体核心工作，负责 STM32CubeMX 工程配置、外部中断程序编写、DWT 按键消抖等关键内容，把控整体实验进度与技术难点。组员承接硬件接线、器件摆放、线路核查等辅助工作；实操过程中遇到不懂的软硬件问题，及时沟通交流，相互配合开展硬件核验、功能测试等配套工作，协助组长推进实验落地，实现档位模拟器正常运行。

实验全部实操结束后，共同研讨实验报告的整体结构、各章节撰写要点、实验数据与故障分析内容，逐项确定报告需要收录的实验原理、操作步骤、实验现象等素材。在统一报告撰写思路之后，由组员汇总所有商讨内容与实验原始资料，完成实验报告全文的内容梳理、文字整理与格式排版工作，完成报告初稿，之后双方互相检查修改，完善最终实验报告。

## 二、实验目的与原理（10 分）

### 2.1 实验目的

1. 掌握 STM32F407 GPIO 外部中断的配置方法；
2. 理解中断服务程序的设计与实现；
3. 学习使用 DWT 计数器实现精准按键消抖；
4. 掌握状态机思想在嵌入式系统中的应用；
5. 培养硬件与软件结合的调试能力。

### 2.2 实验原理

1. 按键按下产生下降沿触发 EXTI 中断；
2. CPU 响应中断，执行档位切换函数；
3. LED 根据档位状态显示不同组合。

### 2.3 汽车电子应用场景

1. 方向盘多功能按键控制；
2. 换挡拨片信号检测；

3. 车载中控物理按键响应。

### 三、硬件设计与连接（15 分）

#### 3.1 硬件连接图



图 1 硬件连接图

#### 3.2 引脚配置说明

| 功能        | GPIO 引脚 | 配置说明                           |
|-----------|---------|--------------------------------|
| LED0 输出   | PF9     | 推挽输出模式，高速驱动 LED                |
| LED1 输出   | PF10    | 推挽输出模式，高速驱动 LED                |
| KEY0 中断输入 | PE4     | 外部中断输入模式，下降沿触发、上拉输入（按键按下时引脚拉低） |

#### 3.3 ISP 烧录电路说明

- BOOT0 = 1，BOOT1 = 0 进入系统存储器（ISP 模式）；
- 通过 USART1（PA9/PA10）接收烧录数据；
- 烧录完成后 BOOT0=0，从 Flash 启动。

## 四、软件设计与实现（30 分）

### 4.1 核心代码流程图

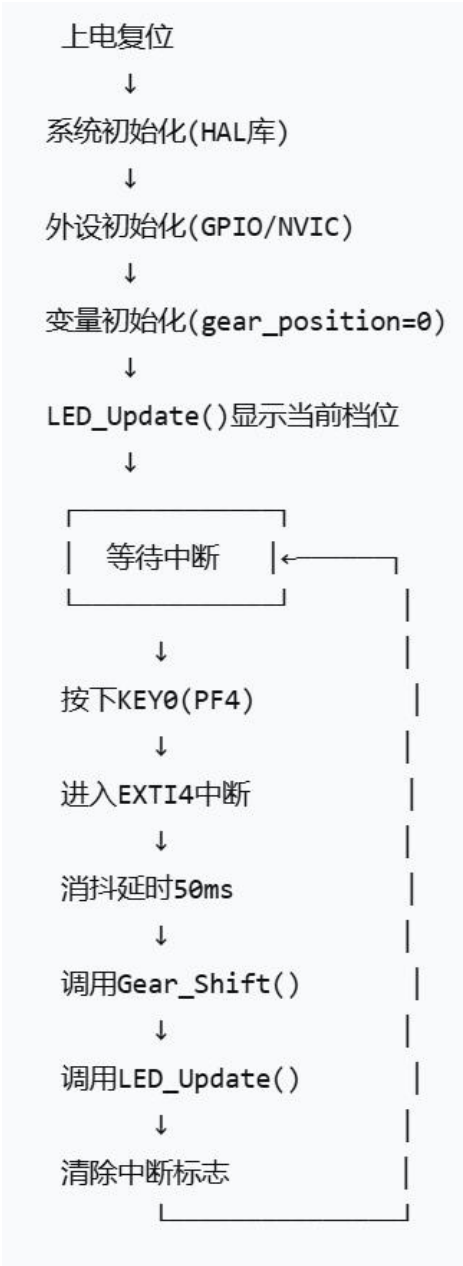


图 2 核心代码流程图

### 4.2 关键代码片段及注释

#### 1. 档位切换函数

```
void Gear_Shift(void) {
```

```

    gear_position++;

    if (gear_position > 2) gear_position = 0;

    if (gear_position != last_gear) {

        last_gear = gear_position;

        LED_Update(); // 更新 LED 状态

    }

}

```

## 2. LED 更新函数

```

void LED_Update(void) {

    switch(gear_position) {

        case 0: // 空档: 两灯全灭

            HAL_GPIO_WritePin(LED0_Pin, GPIO_PIN_RESET);

            HAL_GPIO_WritePin(LED1_Pin, GPIO_PIN_RESET);

            break;

        case 1: // 1 档: LED0 亮

            HAL_GPIO_WritePin(LED0_Pin, GPIO_PIN_SET);

            HAL_GPIO_WritePin(LED1_Pin, GPIO_PIN_RESET);

            break;

        case 2: // 2 档: LED1 亮

            HAL_GPIO_WritePin(LED0_Pin, GPIO_PIN_RESET);

            HAL_GPIO_WritePin(LED1_Pin, GPIO_PIN_SET);

            break;

    }

}

```

}

3. 中断回调函数

```
void HAL_GPIO_EXTI_Callback(uint16_t GPIO_Pin) {  
    if(GPIO_Pin == KEY0_Pin) {  
        HAL_Delay(50); // 软件消抖  
  
        if(HAL_GPIO_ReadPin(KEY0_GPIO_Port, KEY0_Pin) == GPIO_PIN_RESET) {  
            Gear_Shift();  
        }  
    }  
}
```

4.3 中断服务程序说明

- 1. 配置为下降沿触发，按键按下瞬间触发中断；
- 2. 在回调函数中加入消抖逻辑，防止一次按下多次触发；
- 3. 中断优先级可根据需要调整。

4.4 团队成员负责的代码模块说明

| 成员姓名 | 负责模块                     | 完成情况 |
|------|--------------------------|------|
| 刘潇霞  | 实现中断回调函数、<br>LED 显示逻辑和调试 | 已完成  |
| 王舒乐  | main.c 主逻辑和变量定义          | 已完成  |

五、实验结果分析（20 分）

5.1 实验现象记录

| 档位  | Gear_position 值 | LED0 (PF9) | LED1 (PF10) |
|-----|-----------------|------------|-------------|
| 空档  | 0               | 灭          | 灭           |
| 1 档 | 1               | 亮          | 灭           |
| 2 档 | 2               | 灭          | 亮           |

5.2 结果照片或视频截图



图 3 实验结果视频截图

- 1. 上电后默认空档，两灯全灭；
- 2. 按一次 KEY0 → 1 档 → LED0 亮；
- 3. 再按一次 KEY0 → 2 档 → LED1 亮；
- 4. 再按一次 KEY0 → 空档 → 两灯全灭；
- 5. 循环切换。

5.3 与预期结果对比分析

完全符合预期，按键响应灵敏，无抖动误触发。

## 六、问题与讨论（15 分）

### 6.1 调试过程中遇到的问题

| 问题现象            | 可能原因                          | 解决方法                     |
|-----------------|-------------------------------|--------------------------|
| 按一次 KEY0 触发多次中断 | 按键机械抖动                        | 添加 50ms 消抖延时             |
| LED 不按预期亮灭      | Gear_Shift 中未调用<br>LED_Update | 在档位变化时调用<br>LED_Update() |

### 6.2 团队协作解决过程

1. 通过微信同步进度
2. 遇到问题先各自排查，再集中讨论并询问 deep seek

### 6.3 个人心得体会

实验让我熟悉状态机实现档位循环逻辑，初期因未做 DWT 消抖出现档位乱跳，补充消抖代码后恢复正常，加深了对汽车电控档位控制原理的理解。

## 七、思考题回答（5 分）

说明：请回答实验任务书中的思考题

1. 为什么按键要配置为下降沿触发而不是上升沿触发？

答：按键通常接法是一端接 GND，另一端接 GPIO 并配置内部上拉。静止时：引脚为高电平（上拉）。按下时：引脚被拉低，产生一个高→低的跳变，即下降沿。释放时：引脚从低回到高，是上升沿。

下降沿触发对应按键按下的瞬间，符合“按下即响应”的直觉；而上升沿对应释放瞬间，响应会有延迟感。另外，下降沿触发可利用内部上拉电阻，无需外部电路，硬件最简单。

2. DWT 消抖与软件延时消抖相比有什么优势？

答：软件延时消抖：简单易实现，但会阻塞 CPU，在延时期间无法执行其他任务（包括其他中断）。延时长度难以精确权衡：太长影响响应，太短消抖不彻底。



DWT 消抖：利用内核的免费计数器实现非阻塞检测。在中断中记录首次触发的时间戳，退出中断；主循环或定时器中再次检查时间差是否大于消抖阈值。

优势：CPU 不阻塞、精度高（可到 1 个时钟周期）、不占用额外定时器。

综上所述，对于实时性要求高的系统，DWT 消抖更优；简单实验用延时消抖足够。

3. 如果要求从 D 档不能直接切换到 R 档（需经过 N 档），如何修改代码？

答：使用有限状态机，禁止直接跳转。伪代码示例：

```
typedef enum { GEAR_R, GEAR_N, GEAR_D } GearState;
GearState current_gear = GEAR_N;

void ShiftUp(void) {    // 升档：D → N → R
    if (current_gear == GEAR_D) current_gear = GEAR_N;
    else if (current_gear == GEAR_N) current_gear = GEAR_R;
    // 若已是 R 则不变
}

void ShiftDown(void) { // 降档：R → N → D
    if (current_gear == GEAR_R) current_gear = GEAR_N;
    else if (current_gear == GEAR_N) current_gear = GEAR_D;
}
```

无论是从 D 试图直接切到 R, 还是从 R 直接切到 D, 都必须经过 N 档。

4. 中断服务程序中为什么要尽量少执行代码？

答：

1) 实时性：中断优先级较高，如果 ISR 执行时间过长，会延迟其他低优先级中断或主循环任务的响应，导致系统实时性下降。

2) 嵌套问题：长时间占用 CPU 可能导致其他更紧急的中断无法及时响应，甚至造成数据丢失。

3) 资源共享：如果在 ISR 中调用不可重入函数或操作共享资源，容易产生竞态条件。

4) 正确做法: ISR 中只做必要操作 (如清除标志、保存数据), 将耗时处理放到主循环或任务中。

5. 本实验的 LED 是低电平点亮还是高电平点亮? 代码中如何体现?

答: 该实验采用高电平点亮, 因为代码中 case1 将 LED0 设为 GPIO\_PIN\_SET 使其亮, LED1 为 RESET 灭。代码中 HAL\_GPIO\_WritePin(LED0\_GPIO\_Port, LED0\_Pin, GPIO\_PIN\_SET) 就是点亮的具体体现。

6. 汽车电子应用题: 在实际汽车中, 档位传感器通常采用什么类型的信号 (数字量/模拟量/总线信号)?

答: 现代汽车档位传感器主要采用以下方式:

1) CAN/LIN 总线信号 (最常见): 档位信息通过控制器局域网 (CAN) 或局部互联网络 (LIN) 发送到仪表和变速箱控制单元。

2) 模拟量: 早期或低成本车型使用滑动变阻器, 输出电压随档位变化。

3) 数字量: 每个档位对应独立开关 (如霍尔传感器), 输出高/低电平, 但线束较多。

结论: 主流是 CAN 总线信号, 具有抗干扰强、重量轻、可传输多信息 (如档位、误操作诊断等) 的优点。

7. 拓展思考: 如果要实现一个包含 S (运动档) 或 L (低速档) 的多档位系统, 应该如何修改状态机设计?

答: 可以将档位定义为一个 枚举状态机, 例如:

```
typedef enum {  
    GEAR_P, GEAR_R, GEAR_N, GEAR_D, GEAR_S, GEAR_L  
} Gear;
```

设计合法跳转表或状态转换函数, 例如:

从 D 按下按钮进入 S, 再按退出 S 回到 D。

从 D 无法直接到 L, 需经过 M (手动模式) 或专门的低速模式按钮。

可以使用状态转换矩阵 (二维数组) 判断每次请求是否合法。

示例代码框架：

```
bool IsValidTransition(Gear from, Gear to) {  
    // 预定义允许的转换  
  
    const bool allowed[6][6] = {  
        // P    R    N    D    S    L  
        {0, 1, 1, 0, 0, 0}, // from P  
        {1, 0, 1, 0, 0, 0}, // from R  
        {1, 1, 0, 1, 0, 0}, // from N  
        {0, 0, 1, 0, 1, 0}, // from D  
        {0, 0, 0, 1, 0, 0}, // from S  
        {0, 0, 0, 1, 0, 0}, // from L  
    };  
  
    return allowed[from][to];  
}
```

这样可扩展性强，易于维护。

## 八、附录：核心代码

### 1. 档位切换函数

```
void Gear_Shift(void) {  
    // 档位加 1，实现升档效果  
  
    gear_position++;  
  
    // 如果档位超过最大值 2（即档位 3），则回到 0（空档），形成循环  
  
    // 假设只有 0、1、2 三个档位  
  
    if (gear_position > 2) {  
        gear_position = 0;  
    }  
  
    // 判断档位是否真的发生了变化（防止重复调用时无意义刷新）  
  
    if (gear_position != last_gear) {  
        // 更新上一次档位记录，供下次比较使用  
  
        last_gear = gear_position;  
  
        // 调用 LED 更新函数，根据新档位点亮对应的 LED  
  
        LED_Update();  
    }  
}
```

### 2. LED 调动函数

```
void LED_Update(void) {  
    // 根据档位值进行分支选择  
    switch(gear_position) {  
        case 0: // 空档  
            // 熄灭 LED0: 输出低电平（假设高电平点亮，则低电平熄灭）  
            // 注意：实际参数应为 HAL_GPIO_WritePin(LED0_GPIO_Port, LED0_Pin,  
GPIO_PIN_RESET)  
            HAL_GPIO_WritePin(LED0_GPIO_Port, LED0_Pin, GPIO_PIN_RESET);  
            // 熄灭 LED1  
            HAL_GPIO_WritePin(LED1_GPIO_Port, LED1_Pin, GPIO_PIN_RESET);  
            break;  
  
        case 1: // 1 档  
            // 点亮 LED0: 输出高电平  
            HAL_GPIO_WritePin(LED0_GPIO_Port, LED0_Pin, GPIO_PIN_SET);  
            // 熄灭 LED1  
            HAL_GPIO_WritePin(LED1_GPIO_Port, LED1_Pin, GPIO_PIN_RESET);  
            break;  
  
        case 2: // 2 档  
            // 熄灭 LED0  
            HAL_GPIO_WritePin(LED0_GPIO_Port, LED0_Pin, GPIO_PIN_RESET);  
            // 点亮 LED1
```

```

        HAL_GPIO_WritePin(LED1_GPIO_Port, LED1_Pin, GPIO_PIN_SET);

        break;
    }
}

```

### 3. 中断回调函数

```

void HAL_GPIO_EXTI_Callback(uint16_t GPIO_Pin) {

    // 判断触发中断的引脚是不是 KEY0 所在的引脚

    if(GPIO_Pin == KEY0_Pin) {

        // 软件消抖：延时 50 毫秒，跳过按键抖动的不稳定期

        // 注意：HAL_Delay 使用 SysTick 中断，在中断回调中调用会阻塞其他中断，

        // 但对于简单实验可以接受。高标准应用建议使用非阻塞消抖。

        HAL_Delay(50);

        // 延时后再次读取 KEY0 的电平，确认是否真的按下（低电平有效）

        // 如果还是低电平，说明是有效按键，而非抖动

        if(HAL_GPIO_ReadPin(KEY0_GPIO_Port, KEY0_Pin) == GPIO_PIN_RESET) {

            // 调用档位切换函数，改变 gear_position 并更新 LED

            Gear_Shift();

        }

        // 可选：清除中断标志（HAL 库已自动清除，通常不需要手动操作）

        // __HAL_GPIO_EXTI_CLEAR_IT(GPIO_Pin);

    }

}

```